http://visualvm.github.io/pluginscenters.html visualVM 插件下载地址

1 内存分配

昨天我们讲了 JVM 的内存结构

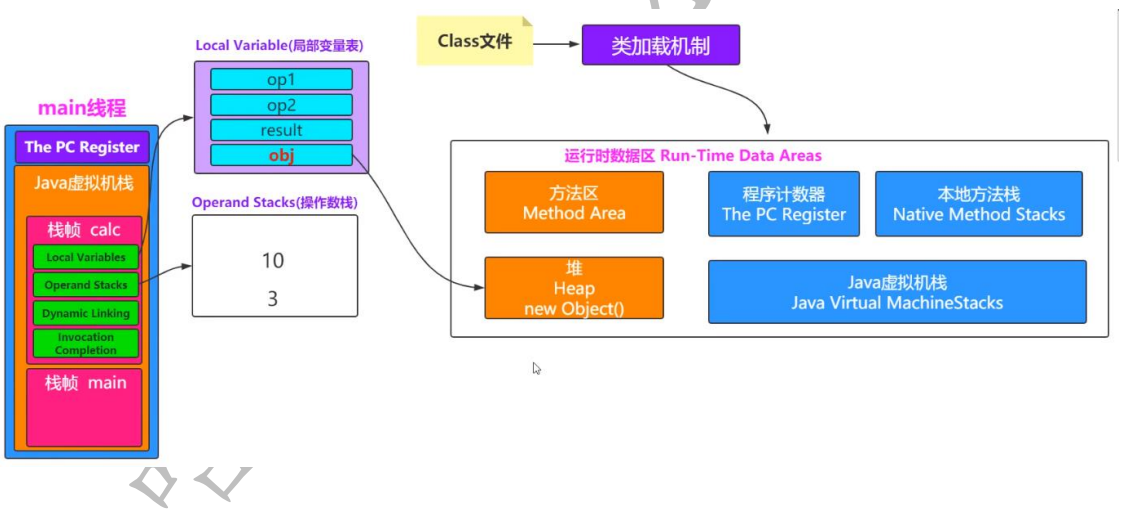
1.1 程序计数器

1.2 本地方法栈

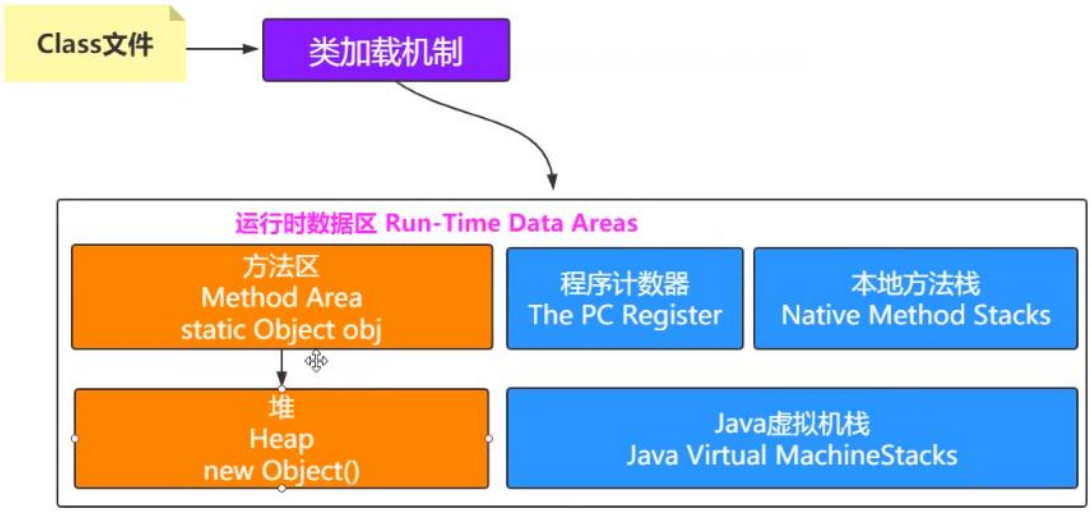
1.3 虚拟机栈

栈帧（Stack Frame）是用于支持虚拟机进行方法调用和方法执行的数据结构，它是虚拟机运行时数据区中的虚拟机栈（Virtual Machine Stack）的栈元素。它包括局部变量表、操作数栈、动态链接、方法返回地址。

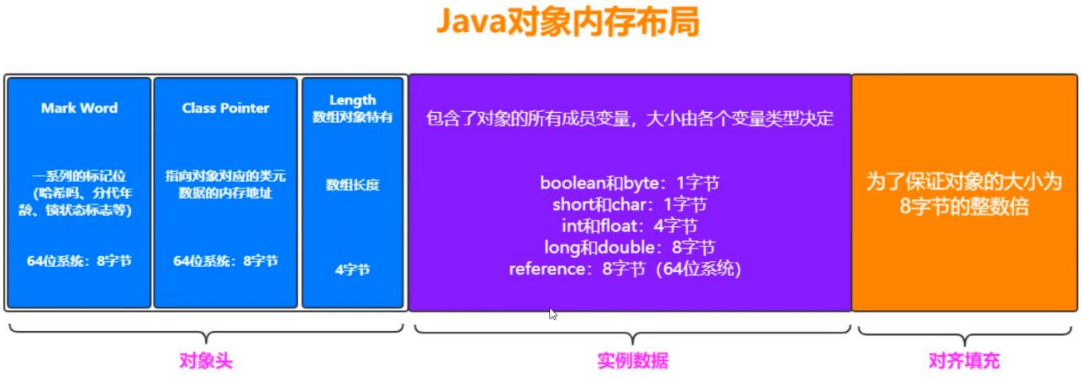
Object obj = new Object()：栈指向堆

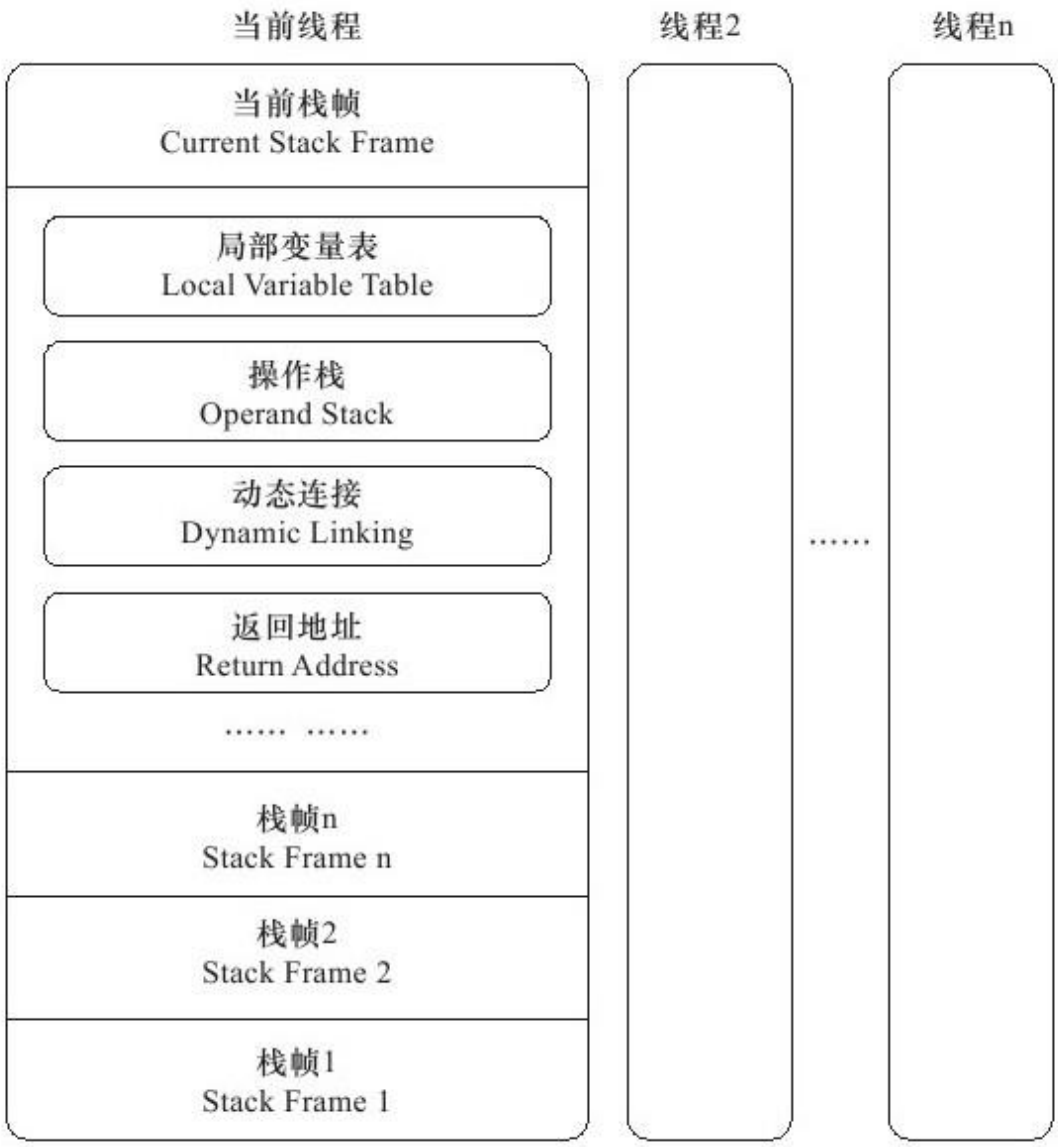


static Object obj = new Object()：方法区指向堆

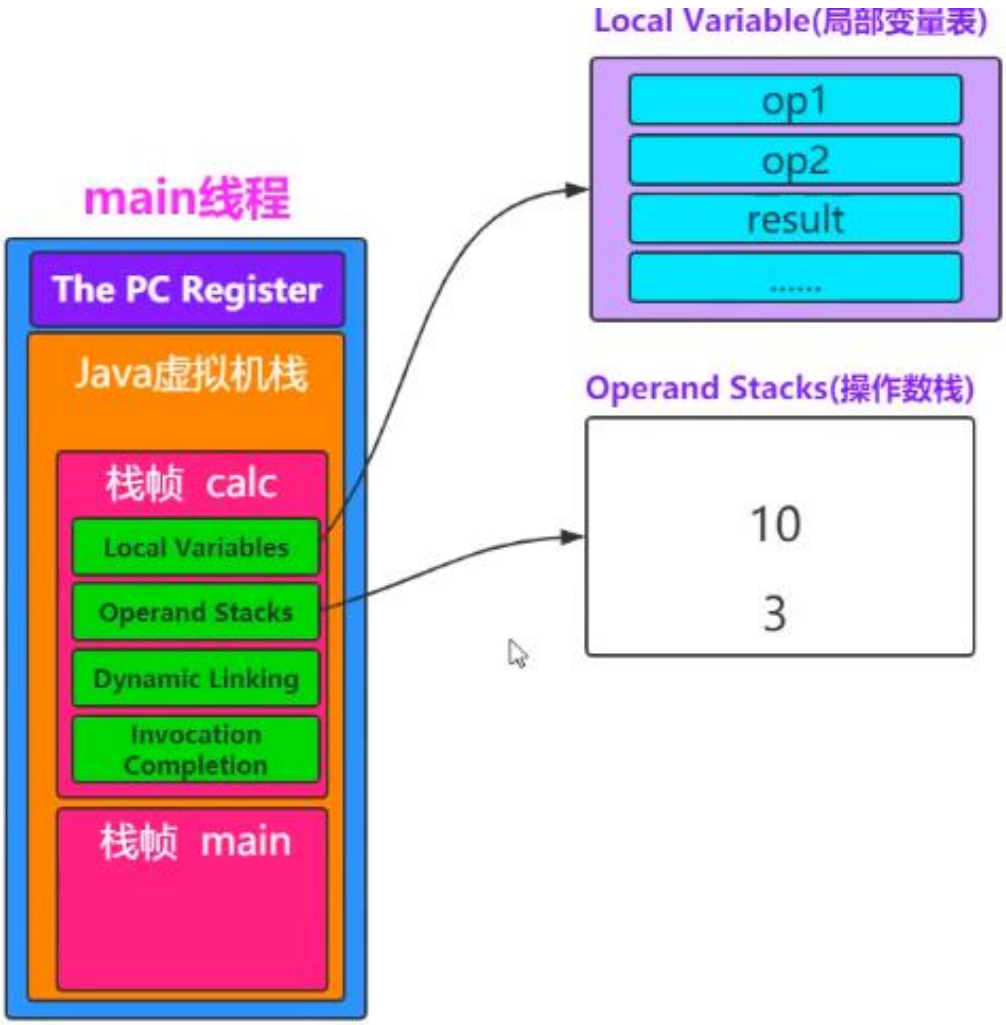


对象中存在指向类信息的类元数据，堆指向方法区





查看反编译字节码：javap -c Person.class >Person.txt




```
public static int calc(int, int);
    Code:
       0: iconst_3 //把常数 3 放入操作数栈
       1: istore_0 //弹出操作数栈中的数据给局部常量表中的第一个
       2: iload_0  //将局部变量表中的第一个数据压入操作数栈
       3: iload_1  //将局部变量表中的第二个数据压入操作数栈
       4: iadd     //对操作数栈中的两个数据相加
       5: istore_2 //弹出操作数栈中的数据给局部常量表中的第三个
       6: iload_2  //将局部变量表中的第三个数据压入操作数栈
       7: ireturn  //返回
```

每一个方法从调用开始至执行完成的过程，都对应着一个栈帧在虚拟机栈里面从入栈到出栈的过程。每一个栈帧都包括了局部变量表、操作数栈、动态连接、方法返回地址和一些额外的附加信息。在编译程序代码的时候，栈帧中需要多大的局部变

量表，多深的操作数栈都已经完全确定了，并且写入到方法表的 Code 属性之中，因此一个栈帧需要分配多少内存，不会受到程序运行期变量数据的影响，而仅仅取决于具体的虚拟机实现。

一个线程中的方法调用链可能会很长，很多方法都同时处于执行状态。对于执行引擎来说，在活动线程中，只有位于栈顶的栈帧才是有效的，称为当前栈帧（Current Stack Frame），与这个栈帧相关联的方法称为当前方法（Current Method）。执行引擎运行的所有字节码指令都只针对当前栈帧进行操作，在概念模型上，典型的栈帧结构如图所示。

1.3.1 局部变量表

局部变量表（Local Variable Table）是一组变量值存储空间，用于存放方法参数和方法内部定义的局部变量。在 Java 程序编译为 Class 文件时，就在方法的 Code 属性的 max_locals 数据项中确定了该方法所需要分配的局部变量表的最大容量。

1.3.2 动态连接

每个栈帧都包含一个指向运行时常量池中该栈帧所属方法的引用，持有这个引用是为了支持方法调用过程中的动态连接（Dynamic Linking）。我们知道 Class 文件的常量池中存有大量的符号引用，字节码中的方法调用指令就以常量池中指向方法的符号引用作为参数。这些符号引用一部分会在类加载阶段或者第一次使用的时候就转化为直接引用，这种转化称为静态解析。另外一部分将在每一次运行期间转化为直接引用，这部分称为动态连接。（静态分派，动态分派）（多态举例）

1.3.3 方法返回地址：

当一个方法开始执行后,只有两种方式可以退出，一种是遇到方法返回的字节码指令；一种是遇见异常，并且这个异常没有在方法体内得到处理。

前面的 3 个都是根据线程的生命周期自动生或灭的！所以我们不需要管，所以只有方法区跟堆才是我们需要考虑 GC 的点！

~~为什么 Jdk1.8 要把方法区从 JVM 里（永久代）移到直接内存（元空间）？~~

1. ~~字符串存在永久代中，容易出现性能问题和内存溢出。~~
2. ~~类及方法的信息等比较难确定其大小，因此对于永久代的大小指定比较困难。~~
3. ~~永久代会为 GC 带来不必要的复杂度，并且回收效率偏低。~~

咸鱼出品 必屬精品

1.4 堆

1.4.1 查看方法区跟堆大小

设置大小

-Xms20M -Xmx20M

打印分配的大小

-XX:+PrintGCDetails

```
long initialMemory = Runtime.getRuntime().totalMemory()/1024/1024;
long maxMemory = Runtime.getRuntime().maxMemory()/1024/1024;
```

默认堆大小：物理电脑内存大小的 1/64 最大的内存大小：物理电脑内存大小的 1/4

1.4.2 新生代跟老年代

那么我们堆里面的内存到底是怎么样的呢？我们一个一个看

首先，我们想到的是堆就预分配了一块内存！！所有的对象都在一块，当我们要进行 GC 的时候，会去找所有的对象是否需要回收！！

但是我们的对象大部分都是朝生夕死的，那么我们就想着，能不能把大部分新创建的对象单独放到某一个地方，减少我们 GC 的范围，那些少部分的非朝生夕死的对象，或者很大的对象放到我们的另外一个地方！！这样我们就有了 2 个区域，old 区跟 young 区！！比例 old 占三分之二，young 占三分之一

年龄：一次垃圾回收，没被回收的对象年龄加 1,年龄大于 15 了就移到老年代。

老年代：对象大小特别大。

这样我们就能保证一般情况下只会对我们的 young 进行 GC，那么这个 GC 的过程叫做 young GC 或者 MinorGC

咕泡出品 必属精品

什么时候进老年代

既然区分了 2 个代，那么老年代肯定也是要放数据的，不然就没用了，那什么时候进入老年代！！新创建的对象在新生代，那么进入老年代肯定是需要条件的！

1.大对象 这个也是可以设置的，目的是为了减少频繁的 youngGC，大对象比较占内存

2.年限达到 15 岁，默认 15，可以设定，但是不能超过 15，**-XX:MaxTenuringThreshold=16** 就会报错，无法创建 JVM，这个是因为 java 的对象头是 4bit 来保存分代年龄

肯定，老年代也是需要去进行内存清理的，那么老年代的 GC 叫做 Major GC，Major GC 一般会伴随至少一次 MinorGC

那么 full GC 就是我的整个堆、元空间等全局范围的 GC

1.4.3Survivor 区域

我们现在分为老年代跟新生代，比我们之前在一起的时候，GC 速率已经快不少了，但是我们又发现了一个问题！！空间碎片，我回收完了之后，有太多碎片，保存不了超过碎片的数据，又会浪费！

所以，又想了个办法，把 young 区再分 Eden 区、From、To 三个区域！那么他们是怎么解决碎片问题的呢

1.我们数据只会存在 Eden 区、from 或者 To 2 个区域的一个区域，也就是我 From.跟 To 肯定有一个是空的！

2.最开始，我们的数据假如都在 Eden 或 From 区，当 Eden 满了或者没有足够的连续空间保存我的对象时，会触发 MinorGC，所有对象的年龄+1，这个时候会把我们 Eden 区的对象赋值到 To 区，之前 From 区的数据就会根据他们的年龄来决定去向，到了去老年代的阀值就去老年代，不然就跟着一起去 To 区！

3.这样，我们的 Eden 区跟 From 区又空了，就不会有碎片，如此循环！解决碎片问题，就是牺牲我一个小区的空间解决空间碎片问题

Eden：from：to=8:1:1

1.4.3.1 对象动态年龄判断

当前放对象的 Survivor 区域里（其中一块区域，放对象的那块 S 区），一批对象的总大小大于这块 Survivor 区域内存大小的 50%（-XX:TargetSurvivorRatio 可以指定），那么此时大于等于这批对象年龄最大值的对象，就可以直接进入老年代了，

黑马出品 必属精品

例如 Survivor 区域里现在有一批对象，年龄 1+年龄 2+年龄 n 的多个年龄对象总和超过了 Survivor 区域的 50%，此时就会把年龄 n（含）以上的对象都放入老年代。这个规则其实是希望那些可能是长期存活的对象，尽早进入老年代。**对象动态年龄判断机制一般是在 minor GC 之后触发**。

1.4.3.2 老年代空间分配担保机制

年轻代每次 minor GC 之前 JVM 都会计算下老年代**剩余可用空间**

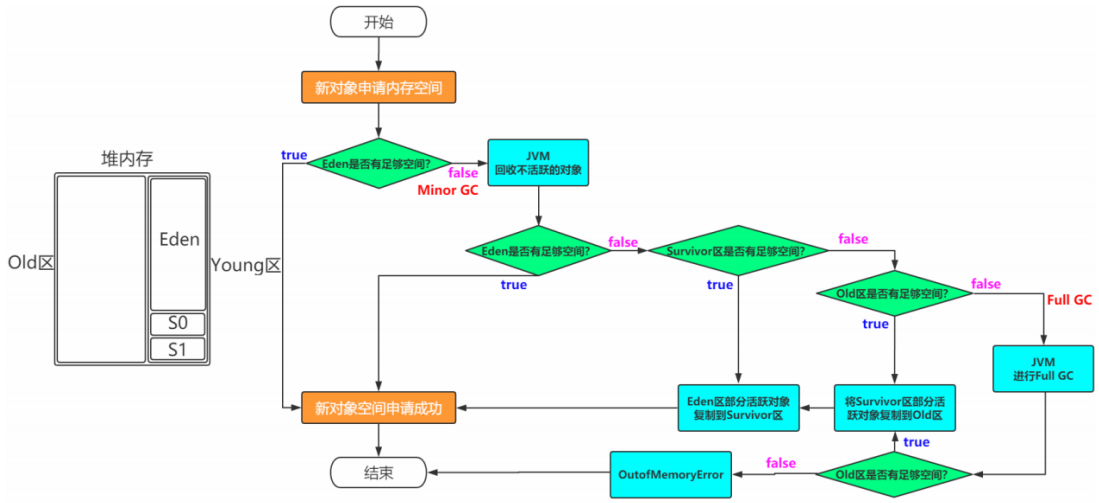
1. 如果这个可用空间小于年轻代里现有的所有对象大小之后（**包括垃圾对象**），就会看一个"-XX:-HandlerPromotionFailure"(jdk1.8 默认是设置了）的参数是否设置了。
2. 如何有这个参数，就会看看老年代的可用内存大小，是否大于之前每一次 minor GC 后进入老年代的对象的平均大小。
3. 如果上一步结果是小于或者之前说的参数没有设置，那么就会触发一次 Full GC，对老年代和年轻代一起回收一次垃圾。
4. 如果回收完还是没有足够空间存放新的对象就会发生"OOM"
5. 当然，如果 minorGC 之后剩余存活的需要挪动到老年代的对象大小还是大于老年代可用空间，那么也会触发 FullGC，FullGC 完之后如果还是没有空间放 minor GC 之后的存活对象，则也会发生"OOM"。

1.4.4 对象一辈子理解

我是一个普通的 Java 对象,我出生在 Eden 区,在 Eden 区我还看到和我长的很像的小兄弟,我们在 Eden 区中玩了挺长时间。有一天 Eden 区中的人实在是太多了,我就被迫去了 Survivor 区的"From"区,自从去了 Survivor 区,我就开始漂了,有时候在 Survivor 的"From"区,有时候在 Survivor 的"To"区,居无定所。直到我 18 岁的时候,爸爸说我成人了,该去社会上闯闯了。

于是我就去了年老代那边,年老代里,人很多,并且年龄都挺大的,我在这里也认识了很多人。在年老代里,我生活了 20 年(每次 GC 加一岁),然后被回收。

老师品 必屬精品



1.4.5 常见问题

- 如何理解 Minor/Major/Full GC

Minor GC:新生代
Major GC:老年代 ，MajorGC 通常会伴随着 MinorGC，也就意味着会触发 FullGC
Full GC:新生代+老年代
FullGC：STW（Stop The World） ，会消耗时间

- 为什么需要 Survivor 区?只有 Eden 不行吗？

如果没有 Survivor,Eden 区每进行一次 Minor GC,并且没有年龄限制的话，存活的对象就会被送到老年代。
这样一来，老年代很快被填满,触发 Major GC(因为 Major GC 一般伴随着 Minor GC,也可以看做触发了 Full GC)。
老年代的内存空间远大于新生代,进行一次 Full GC 消耗的时间比 Minor GC 长得多。
执行时间长有什么坏处?频发的 Full GC 消耗的时间很长,会影响大型程序的执行和响应速度。

可能你会说，那就对老年代的空间进行增加或者较少咯。
假如增加老年代空间，更多存活对象才能填满老年代。虽然降低 Full GC 频率，但是随着老年代空间加大,一旦发生 Full
GC,执行所需要的时间更长。
假如减少老年代空间，虽然 Full GC 所需时间减少，但是老年代很快被存活对象填满,Full GC 频率增加。

所以 Survivor 的存在意义,就是减少被送到老年代的对象,进而减少 Full GC 的发生,Survivor 的预筛选保证,只有经历 16 次 Minor GC 还能在新生代中存活的对象,才会被送到老年代。

- 为什么需要两个 Survivor 区？

最大的好处就是解决了碎片化。也就是说为什么一个 Survivor 区不行?第一部分中,我们知道了必须设置 Survivor 区。假设现在只有一个 Survivor 区,我们来模拟一下流程:刚刚新建的对象在 Eden 中,一旦 Eden 满了,触发一次 Minor GC,Eden 中的存活对象就会被移动到 Survivor 区。这样继续循环下去,下一次 Eden 满了的时候,问题来了,此时进行 Minor GC,Eden 和 Survivor 各有一些存活对象,如果此时把 Eden 区的存活对象硬放到 Survivor 区,很明显这两部分对象所占有的内存是不连续的,也就导致了内存碎片化。永远有一个 Survivor space 是空的,另一个非空的 Survivor space 无碎片。

- 新生代中 Eden:S0:S1 为什么是 8:1:1?

首先 S0 和 S1 必须一样大，不一样大的话，复制的时候会出问题
第二个为啥 Eden 区要占大部分，因为我们对象大部分都是“朝生夕死”，对象的生命周期比较短，然后直接可以在 Eden 去创建，如果 Eden 区小的话，会经常触发 minorGC

1.5 方法区：永久代

1.6 内存区域的一个问题

```
public class test{
    public int i = 10;//i 作为成员变量在类加载的时候会存在方法区存字段的描述信息，10 作为字面量会保存在方法区的常量池中。当 new Test 对象只要，i 和 10 会随着对象一起保存在堆中。
    public static int j = 11;//j 作为静态变量，会保存在方法区中，11 作为字面量会保存在常量池中

    public String s1 = "123";//s1 作为成员变量在类加载的时候会存在方法区存字段的描述信息，“123”作为字面量会保存在方法区的常量池中。当 new Test 对象时，s1 以及“123”的真实地址会随着对象一起保存在堆中。

    public void add(){
        int k = 12;//k 作为基本数据类型，会被存在虚拟机栈中，12 作为字面量会被存在常量池中。
        String s2 = "321";//s2 作为引用类型，会被存在虚拟机栈中，“321”作为字面量会被存在常量池中。
    }

}
```

哈泡出品 必屠精品

2 体验与验证

2.1 堆内存溢出

2.1.1 代码

```
@RestController
public class HeapController {
    List<Person> list=new ArrayList<Person>();
    @GetMapping("/heap")
    public String heap() throws Exception{
        while(true){
            list.add(new Person());
            Thread.sleep(1);
        }
    }
}
```

记得设置参数比如-Xmx20M -Xms20M

2.1.2 运行结果

访问->http://localhost:8080/heap

```
Exception in thread "http-nio-8080-exec-2" java.lang.OutOfMemoryError:
GC overhead limit exceeded
```

2.2 方法区内存溢出

比如向方法区中添加 Class 的信息

2.2.1asm 依赖和 Class 代码

```
<dependency>
    <groupId>asm</groupId>
    <artifactId>asm</artifactId>
    <version>3.3.1</version>
</dependency>
```

```
public class MyMetaspace extends ClassLoader {
    public static List<Class<?>> createClasses() {
        List<Class<?>> classes = new ArrayList<Class<?>>();
        for (int i = 0; i < 10000000; ++i) {
            ClassWriter cw = new ClassWriter(0);
            cw.visit(Opcodes.V1_1, Opcodes.ACC_PUBLIC, "Class" + i, nul
l,
                     "java/lang/Object", null);
            MethodVisitor mw = cw.visitMethod(Opcodes.ACC_PUBLIC, "<ini
t>",
                                              "()V", null, null);
            mw.visitVarInsn(Opcodes.ALOAD, 0);
```

哈包出品 必属精品

```
            mw.visitMethodInsn(Opcodes.INVOKESPECIAL, "java/lang/Object
",
                                "<init>", "()V");
            mw.visitInsn(Opcodes.RETURN);
            mw.visitMaxs(1, 1); mw.visitEnd();
            Metaspace test = new Metaspace();
            byte[] code = cw.toByteArray();
            Class<?> exampleClass = test.defineClass("Class" + i, code,
 0, code.length);
            classes.add(exampleClass);
        }
        return classes;
    }
}
```

2.2.2 代码

```
@RestController
public class NonHeapController {
    List<Class<?>> list=new ArrayList<Class<?>>();
    @GetMapping("/nonheap")
    public String nonheap() throws Exception{
        while(true){
            list.addAll(MyMetaspace.createClasses());
            Thread.sleep(5);
        }
    }
}
```

设置 Metaspace 的大小，比如-XX:MetaspaceSize=50M -XX:MaxMetaspaceSize=50M

2.2.3 运行结果

访问->http://localhost:8080/nonheap

```
java.lang.OutOfMemoryError: Metaspace
    at java.lang.ClassLoader.defineClass1(Native Method) ~[na:1.8.0_191]

    at java.lang.ClassLoader.defineClass(ClassLoader.java:763) ~[na:1.8.
0_191]
```

2.3 虚拟机栈内存溢出

2.3.1 代码演示 StackOverFlow

```
public class StackDemo {
    public static long count=0;
    public static void method(long i){
        System.out.println(count++);
        method(i); }
    public static void main(String[] args) {
```

哈泽出品 必属精品

```
        method(1);
    }
}
```

2.3.2 运行结果

```
7252
7253
7254
7255
Exception in thread "main" java.lang.StackOverflowError
    at sun.nio.cs.UTF_8$Encoder.encodeLoop(UTF_8.java:691)
    at java.nio.charset.CharsetEncoder.encode(CharsetEncoder.java:579)
```

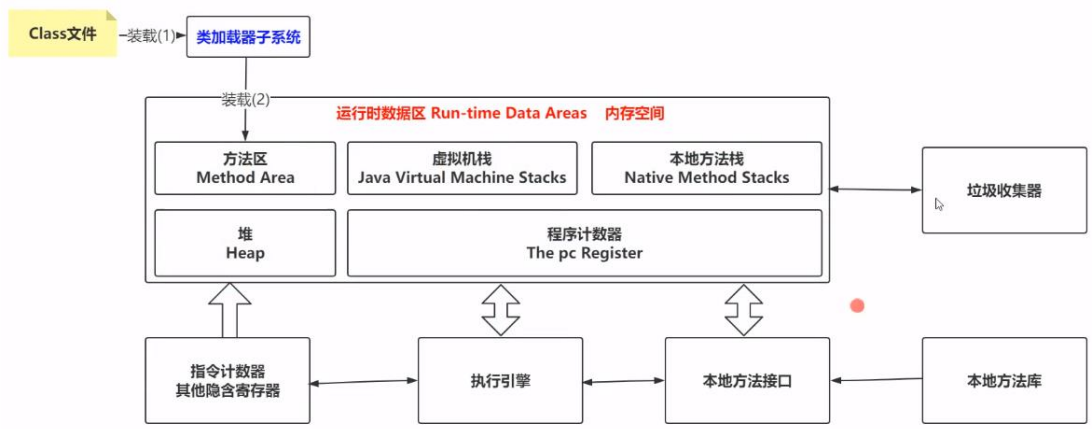
2.3.3 理解和说明

Stack Space 用来做方法的递归调用时压入 Stack Frame(栈帧)。所以当递归调用太深的时候，就有可能耗尽 Stack Space，爆出 StackOverflow 的错误。

-Xss128k：设置每个线程的堆栈大小。JDK 5 以后每个线程堆栈大小为 1M，以前每个线程堆栈大小为 256K。根据应用的线 程所需内存大小进行调整。在相同物理内存下，减小这个值能生成更多的线程。但是操作系统对一个进程内的线程数还是有 限制的，不能无限生成，经验值在 3000~5000 左右。

线程栈的大小是个双刃剑，如果设置过小，可能会出现栈溢出，特别是在该线程内有递归、大的循环时出现溢出的可能性更 大，如果该值设置过大，就有影响到创建栈的数量，如果是多线程的应用，就会出现内存溢出的错误。

3 JVM 全貌



聊完了内存区域，接下来要聊垃圾回收了

4 补充

4.1 字节码指令

指令码	助记符	说明
0x00	nop	无操作
0x01	aconst_null	将 null 推送至栈顶
0x02	iconst_m1	将 int 型-1 推送至栈顶
0x03	iconst_0	将 int 型 0 推送至栈顶
0x04	iconst_1	将 int 型 1 推送至栈顶
0x05	iconst_2	将 int 型 2 推送至栈顶
0x06	iconst_3	将 int 型 3 推送至栈顶
0x07	iconst_4	将 int 型 4 推送至栈顶
0x08	iconst_5	将 int 型 5 推送至栈顶
0x09	lconst_0	将 long 型 0 推送至栈顶
0x0a	lconst_1	将 long 型 1 推送至栈顶
0x0b	fconst_0	将 float 型 0 推送至栈顶
0x0c	fconst_1	将 float 型 1 推送至栈顶
0x0d	fconst_2	将 float 型 2 推送至栈顶
0x0e	dconst_0	将 double 型 0 推送至栈顶
0x0f	dconst_1	将 double 型 1 推送至栈顶
0x10	bipush	将单字节的常量值(-128~127)推送至栈顶
0x11	sipush	将一个短整型常量值(-32768~32767)推送至栈顶
0x12	ldc	将 int, float 或 String 型常量值从常量池中推送至栈顶
0x13	ldc_w	将 int, float 或 String 型常量值从常量池中推送至栈顶（宽索引）
0x14	ldc2_w	将 long 或 double 型常量值从常量池中推送至栈顶（宽索引）
0x15	iload	将指定的 int 型本地变量推送至栈顶
0x16	lload	将指定的 long 型本地变量推送至栈顶
0x17	fload	将指定的 float 型本地变量推送至栈顶
0x18	dload	将指定的 double 型本地变量推送至栈顶
0x19	aload	将指定的引用类型本地变量推送至栈顶
0x1a	iload_0	将第一个 int 型本地变量推送至栈顶
0x1b	iload_1	将第二个 int 型本地变量推送至栈顶
0x1c	iload_2	将第三个 int 型本地变量推送至栈顶
0x1d	iload_3	将第四个 int 型本地变量推送至栈顶
0x1e	lload_0	将第一个 long 型本地变量推送至栈顶
0x1f	lload_1	将第二个 long 型本地变量推送至栈顶
0x20	lload_2	将第三个 long 型本地变量推送至栈顶
0x21	lload_3	将第四个 long 型本地变量推送至栈顶
0x22	fload_0	将第一个 float 型本地变量推送至栈顶
0x23	fload_1	将第二个 float 型本地变量推送至栈顶
0x24	fload_2	将第三个 float 型本地变量推送至栈顶
0x25	fload_3	将第四个 float 型本地变量推送至栈顶
0x26	dload_0	将第一个 double 型本地变量推送至栈顶
0x27	dload_1	将第二个 double 型本地变量推送至栈顶

咕泡出品 必属精品

0x28 dload_2 将第三个 double 型本地变量推送至栈顶
0x29 dload_3 将第四个 double 型本地变量推送至栈顶
0x2a aload_0 将第一个引用类型本地变量推送至栈顶
0x2b aload_1 将第二个引用类型本地变量推送至栈顶
0x2c aload_2 将第三个引用类型本地变量推送至栈顶
0x2d aload_3 将第四个引用类型本地变量推送至栈顶
0x2e iaload 将 int 型数组指定索引的值推送至栈顶
0x2f laload 将 long 型数组指定索引的值推送至栈顶
0x30 faload 将 float 型数组指定索引的值推送至栈顶
0x31 daload 将 double 型数组指定索引的值推送至栈顶
0x32 aaload 将引用型数组指定索引的值推送至栈顶
0x33 baload 将 boolean 或 byte 型数组指定索引的值推送至栈顶
0x34 caload 将 char 型数组指定索引的值推送至栈顶
0x35 saload 将 short 型数组指定索引的值推送至栈顶
0x36 istore 将栈顶 int 型数值存入指定本地变量
0x37 lstore 将栈顶 long 型数值存入指定本地变量
0x38 fstore 将栈顶 float 型数值存入指定本地变量
0x39 dstore 将栈顶 double 型数值存入指定本地变量
0x3a astore 将栈顶引用型数值存入指定本地变量
0x3b istore_0 将栈顶 int 型数值存入第一个本地变量
0x3c istore_1 将栈顶 int 型数值存入第二个本地变量
0x3d istore_2 将栈顶 int 型数值存入第三个本地变量
0x3e istore_3 将栈顶 int 型数值存入第四个本地变量
0x3f lstore_0 将栈顶 long 型数值存入第一个本地变量
0x40 lstore_1 将栈顶 long 型数值存入第二个本地变量
0x41 lstore_2 将栈顶 long 型数值存入第三个本地变量
0x42 lstore_3 将栈顶 long 型数值存入第四个本地变量
0x43 fstore_0 将栈顶 float 型数值存入第一个本地变量
0x44 fstore_1 将栈顶 float 型数值存入第二个本地变量
0x45 fstore_2 将栈顶 float 型数值存入第三个本地变量
0x46 fstore_3 将栈顶 float 型数值存入第四个本地变量
0x47 dstore_0 将栈顶 double 型数值存入第一个本地变量
0x48 dstore_1 将栈顶 double 型数值存入第二个本地变量
0x49 dstore_2 将栈顶 double 型数值存入第三个本地变量
0x4a dstore_3 将栈顶 double 型数值存入第四个本地变量
0x4b astore_0 将栈顶引用型数值存入第一个本地变量
0x4c astore_1 将栈顶引用型数值存入第二个本地变量
0x4d astore_2 将栈顶引用型数值存入第三个本地变量
0x4e astore_3 将栈顶引用型数值存入第四个本地变量
0x4f iastore 将栈顶 int 型数值存入指定数组的指定索引位置
0x50 lastore 将栈顶 long 型数值存入指定数组的指定索引位置
0x51 fastore 将栈顶 float 型数值存入指定数组的指定索引位置
0x52 dastore 将栈顶 double 型数值存入指定数组的指定索引位置
0x53 aastore 将栈顶引用型数值存入指定数组的指定索引位置
0x54 bastore 将栈顶 boolean 或 byte 型数值存入指定数组的指定索引位置

若泉出品 必属精品

0x55 castore 将栈顶 char 型数值存入指定数组的指定索引位置
0x56 sastore 将栈顶 short 型数值存入指定数组的指定索引位置
0x57 pop 将栈顶数值弹出 (数值不能是 long 或 double 类型的)
0x58 pop2 将栈顶的一个（long 或 double 类型的)或两个数值弹出（其它）
0x59 dup 复制栈顶数值并将复制值压入栈顶
0x5a dup_x1 复制栈顶数值并将两个复制值压入栈顶
0x5b dup_x2 复制栈顶数值并将三个（或两个）复制值压入栈顶
0x5c dup2 复制栈顶一个（long 或 double 类型的)或两个（其它）数值并将复制值压入栈顶
0x5d dup2_x1 复制栈顶的一个或两个值，将其插入栈顶那两个或三个值的下面
0x5e dup2_x2 复制栈顶的一个或两个值，将其插入栈顶那两个、三个或四个值的下面
0x5f swap 将栈最顶端的两个数值互换(数值不能是 long 或 double 类型的)
0x60 iadd 将栈顶两 int 型数值相加并将结果压入栈顶
0x61 ladd 将栈顶两 long 型数值相加并将结果压入栈顶
0x62 fadd 将栈顶两 float 型数值相加并将结果压入栈顶
0x63 dadd 将栈顶两 double 型数值相加并将结果压入栈顶
0x64 isub 将栈顶两 int 型数值相减并将结果压入栈顶
0x65 lsub 将栈顶两 long 型数值相减并将结果压入栈顶
0x66 fsub 将栈顶两 float 型数值相减并将结果压入栈顶
0x67 dsub 将栈顶两 double 型数值相减并将结果压入栈顶
0x68 imul 将栈顶两 int 型数值相乘并将结果压入栈顶
0x69 lmul 将栈顶两 long 型数值相乘并将结果压入栈顶
0x6a fmul 将栈顶两 float 型数值相乘并将结果压入栈顶
0x6b dmul 将栈顶两 double 型数值相乘并将结果压入栈顶
0x6c idiv 将栈顶两 int 型数值相除并将结果压入栈顶
0x6d ldiv 将栈顶两 long 型数值相除并将结果压入栈顶
0x6e fdiv 将栈顶两 float 型数值相除并将结果压入栈顶
0x6f ddiv 将栈顶两 double 型数值相除并将结果压入栈顶
0x70 irem 将栈顶两 int 型数值作取模运算并将结果压入栈顶
0x71 lrem 将栈顶两 long 型数值作取模运算并将结果压入栈顶
0x72 frem 将栈顶两 float 型数值作取模运算并将结果压入栈顶
0x73 drem 将栈顶两 double 型数值作取模运算并将结果压入栈顶
0x74 ineg 将栈顶 int 型数值取负并将结果压入栈顶
0x75 lneg 将栈顶 long 型数值取负并将结果压入栈顶
0x76 fneg 将栈顶 float 型数值取负并将结果压入栈顶
0x77 dneg 将栈顶 double 型数值取负并将结果压入栈顶
0x78 ishl 将 int 型数值左移位指定位数并将结果压入栈顶
0x79 lshl 将 long 型数值左移位指定位数并将结果压入栈顶
0x7a ishr 将 int 型数值右（符号）移位指定位数并将结果压入栈顶
0x7b lshr 将 long 型数值右（符号）移位指定位数并将结果压入栈顶
0x7c iushr 将 int 型数值右（无符号）移位指定位数并将结果压入栈顶
0x7d lushr 将 long 型数值右（无符号）移位指定位数并将结果压入栈顶
0x7e iand 将栈顶两 int 型数值作“按位与”并将结果压入栈顶
0x7f land 将栈顶两 long 型数值作“按位与”并将结果压入栈顶

咕泡出品必属精品

0x80 ior 将栈顶两 int 型数值作“按位或”并将结果压入栈顶
0x81 lor 将栈顶两 long 型数值作“按位或”并将结果压入栈顶
0x82 ixor 将栈顶两 int 型数值作“按位异或”并将结果压入栈顶
0x83 lxor 将栈顶两 long 型数值作“按位异或”并将结果压入栈顶
0x84 iinc 将指定 int 型变量增加指定值（i++, i--, i+=2）
0x85 i2l 将栈顶 int 型数值强制转换成 long 型数值并将结果压入栈顶
0x86 i2f 将栈顶 int 型数值强制转换成 float 型数值并将结果压入栈顶
0x87 i2d 将栈顶 int 型数值强制转换成 double 型数值并将结果压入栈顶
0x88 l2i 将栈顶 long 型数值强制转换成 int 型数值并将结果压入栈顶
0x89 l2f 将栈顶 long 型数值强制转换成 float 型数值并将结果压入栈顶
0x8a l2d 将栈顶 long 型数值强制转换成 double 型数值并将结果压入栈顶
0x8b f2i 将栈顶 float 型数值强制转换成 int 型数值并将结果压入栈顶
0x8c f2l 将栈顶 float 型数值强制转换成 long 型数值并将结果压入栈顶
0x8d f2d 将栈顶 float 型数值强制转换成 double 型数值并将结果压入栈顶
0x8e d2i 将栈顶 double 型数值强制转换成 int 型数值并将结果压入栈顶
0x8f d2l 将栈顶 double 型数值强制转换成 long 型数值并将结果压入栈顶
0x90 d2f 将栈顶 double 型数值强制转换成 float 型数值并将结果压入栈顶
0x91 i2b 将栈顶 int 型数值强制转换成 byte 型数值并将结果压入栈顶
0x92 i2c 将栈顶 int 型数值强制转换成 char 型数值并将结果压入栈顶
0x93 i2s 将栈顶 int 型数值强制转换成 short 型数值并将结果压入栈顶
0x94 lcmp 比较栈顶两 long 型数值大小，并将结果（1，0，-1）压入栈顶
0x95 fcmpl 比较栈顶两 float 型数值大小，并将结果（1，0，-1）压入栈顶；当其中一个数值为 NaN 时，将-1 压入栈顶
0x96 fcmpg 比较栈顶两 float 型数值大小，并将结果（1，0，-1）压入栈顶；当其中一个数值为 NaN 时，将 1 压入栈顶
0x97 dcmpl 比较栈顶两 double 型数值大小，并将结果（1，0，-1）压入栈顶；当其中一个数值为 NaN 时，将-1 压入栈顶
0x98 dcmpg 比较栈顶两 double 型数值大小，并将结果（1，0，-1）压入栈顶；当其中一个数值为 NaN 时，将 1 压入栈顶
0x99 ifeq 当栈顶 int 型数值等于 0 时跳转
0x9a ifne 当栈顶 int 型数值不等于 0 时跳转
0x9b iflt 当栈顶 int 型数值小于 0 时跳转
0x9c ifge 当栈顶 int 型数值大于等于 0 时跳转
0x9d ifgt 当栈顶 int 型数值大于 0 时跳转
0x9e ifle 当栈顶 int 型数值小于等于 0 时跳转
0x9f if_icmpeq 比较栈顶两 int 型数值大小，当结果等于 0 时跳转
0xa0 if_icmpne 比较栈顶两 int 型数值大小，当结果不等于 0 时跳转
0xa1 if_icmplt 比较栈顶两 int 型数值大小，当结果小于 0 时跳转
0xa2 if_icmpge 比较栈顶两 int 型数值大小，当结果大于等于 0 时跳转
0xa3 if_icmpgt 比较栈顶两 int 型数值大小，当结果大于 0 时跳转
0xa4 if_icmple 比较栈顶两 int 型数值大小，当结果小于等于 0 时跳转
0xa5 if_acmpeq 比较栈顶两引用型数值，当结果相等时跳转
0xa6 if_acmpne 比较栈顶两引用型数值，当结果不相等时跳转
0xa7 **goto** 无条件跳转
0xa8 jsr 跳转至指定 16 位 offset 位置，并将 jsr 下一条指令地址压入栈顶

0xa9 ret　　返回至本地变量指定的 index 的指令位置（一般与 jsr，jsr_w 联合使用）
0xaa tableswitch　　用于 switch 条件跳转，case 值连续（可变长度指令）
0xab lookupswitch　　用于 switch 条件跳转，case 值不连续（可变长度指令）
0xac ireturn　　从当前方法返回 int
0xad lreturn　　从当前方法返回 long
0xae freturn　　从当前方法返回 float
0xaf dreturn　　从当前方法返回 double
0xb0 areturn　　从当前方法返回对象引用
0xb1 return　　从当前方法返回 void
0xb2 getstatic　　获取指定类的静态域，并将其值压入栈顶
0xb3 putstatic　　为指定的类的静态域赋值
0xb4 getfield　　获取指定类的实例域，并将其值压入栈顶
0xb5 putfield　　为指定的类的实例域赋值
0xb6 invokevirtual　　调用实例方法
0xb7 invokespecial　　调用超类构造方法，实例初始化方法，私有方法
0xb8 invokestatic　　调用静态方法
0xb9 invokeinterface 调用接口方法
0xba invokedynamic　调用动态链接方法
0xbb new　　创建一个对象，并将其引用值压入栈顶
0xbc newarray　　创建一个指定原始类型（如 int，float，char…）的数组，并将其引用值压入栈顶
0xbd anewarray　　创建一个引用型（如类，接口，数组）的数组，并将其引用值压入栈顶
0xbe arraylength 获得数组的长度值并压入栈顶
0xbf athrow　　将栈顶的异常抛出
0xc0 checkcast　　检验类型转换，检验未通过将抛出 ClassCastException
0xc1 instanceof 检验对象是否是指定的类的实例，如果是将 1 压入栈顶，否则将 0 压入栈顶
0xc2 monitorenter　　获得对象的锁，用于同步方法或同步块
0xc3 monitorexit　　释放对象的锁，用于同步方法或同步块
0xc4 wide　　扩大本地变量索引的宽度
0xc5 multianewarray 创建指定类型和指定维度的多维数组（执行该指令时，操作栈中必须包含各维度的长度值），并将其引用值压入栈顶
0xc6 ifnull　　为 null 时跳转
0xc7 ifnonnull　　不为 null 时跳转
0xc8 goto_w　　无条件跳转
0xc9 jsr_w　　跳转至指定 32 位 offset 位置，并将 jsr_w 下一条指令地址压入栈顶
==========
0xca breakpoint　调试时的断点标记
0xfe impdep1　　为特定软件而预留的语言后门
0xff impdep2　　为特定硬件而预留的语言后门
最后三个为保留指令

此为出品 必属精品

咕泡出品 必屬精品